

Mathematics for Drone Piloting

Loke Strøm Haugsnes

May 8, 2026

This document describes the mathematical model used for rigid body dynamics in the drone piloting simulation.

1 Rigid body

Definitions:

M is the mass of the rigid body.
 P is the position of the rigid body.
 v is the velocity of the rigid body.
 a is the acceleration of the rigid body.
 O_M is the center of mass of the rigid body in body frame.
 I is the inertia tensor of the rigid body about its center of mass.
 L is the angular momentum of the rigid body.
 q is the orientation of the rigid body, represented as a quaternion.
 τ is the torque applied to the rigid body.
 F_S is the sum of the forces applied to the rigid body in body frame.
 F_{DS} is the sum of the force displacement from O_M .

The subscript B denotes that the variable is expressed in the body frame.

Variable Initialization:

From the drone configuration file:

M is defined as the mass of the drone.
 I_B is defined as the inertia tensor of the drone.
 O_M is defined as the center of mass of the drone.

Note: Other variables in the JSON configuration file have no effect on the math, as they are only used for visualization and other purposes even the motor properties, as they are used to calculate the forces applied to the drone, which is represented by F_S and F_{DS} .

Then the rest of the variables will be initialized too zero, as the drone starts

at rest. Or they may be affected by the map configuration file or other factors like swapping the drone mid flight. This covers all the variables initialized state. The next step is to define how these variables are updated during the simulation. But before that it is important to note that the sum of forces F_S and the sum of force displacement F_{DS} is the basis for all the other variables. It will be updated based on the inputs of users and the physics of the environment. That again comes from the drone control script and the map configuration file.

Equations of motion:

The equations of motion for the rigid body are given by:

$$\begin{aligned}\ddot{P} &= \frac{F_S}{M} \\ \tau &= F_S \times F_{DS} \\ \dot{L} &= \tau_B \\ \omega &= I^{-1}L \\ \dot{q} &= \Omega(\omega)\end{aligned}$$

where:

$\Omega(\omega)$ is a axis quaternion ($||\omega||, |\omega|$)

2 PID controller

Additional Definitions:

P_d is the desired position in world frame.
 v_d is the desired velocity in world frame.
 q_d is the desired orientation.
 ω_d is the desired angular velocity.
 $e_P = P_d - P$ is the position error.
 $e_v = v_d - v$ is the velocity error.
 e_q is the orientation error (quaternion difference).
 $e_\omega = \omega_d - \omega$ is the angular velocity error.
 F_W is the total force in world frame.
 $R(q, v)$ rotates the vector v by the quaternion q .
 g is gravitational acceleration vector.

PID Structure:

Control is separated into translational and rotational components.

Translational Control (Position $\rightarrow$ Force):

The desired total force in world frame is:

$$F_W = K_p^P e_P + K_d^P e_v + K_i^P \int e_P dt + Mg$$

This force is transformed into body frame:

$$F_S = R(q', F_W)$$

Rotational Control (Orientation $\rightarrow$ Torque):

The orientation error is computed using quaternion difference:

$$e_q = q_d \cdot q^{-1}$$

The torque is defined by:

$$\tau_B = K_p^R \text{vec}(e_q) + K_d^R e_\omega + K_i^R \int \text{vec}(e_q) dt$$

where $\text{vec}(e_q)$ extracts the vector part of the quaternion.

This is just an example actual implementations may use more PID controllers or even just PI/PD or other combinations.

Force and Torque Allocation:

The total force and torque are given by:

$$F_S = \sum_{i=1}^n F_i$$
$$\tau = \sum_{i=1}^n D_i \times F_i$$

where:

F_i is the force produced by the i -th motor.

D_i is the position of the i -th motor relative to the center of mass.

This can be written as a linear system:

$$\begin{bmatrix} F_S \\ \tau \end{bmatrix} = \begin{bmatrix} \mathbb{1}_3 & \mathbb{1}_3 & \cdots & \mathbb{1}_3 \\ [D_1]_\times & [D_2]_\times & \cdots & [D_n]_\times \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ \vdots \\ F_n \end{bmatrix}$$

Final Control Pipeline:

At each timestep:

1. Compute position and velocity error.
2. Compute desired force F_S .
3. Compute orientation and angular velocity error.
4. Compute torque τ_B .
5. Solve for motor forces F_i .
6. Apply forces to update F_S and F_{DS} .

3 References

This math is based on the more in-depth document in the rocket simulation project, available at the following link: <https://github.com/Lokestrom/rocketSim-code/blob/main/math.pdf>